

From Fragmentation to Systematic Design: Architecting LLM-Based Multi-Agent Systems

Caishen Zhou¹, Yihong Tang¹, Kehai Chen¹, Xuefeng Bai¹, Shuhan Qi¹, Li Shen¹, and Min Zhang¹

¹Affiliation not available

February 17, 2026

Abstract

The integration of Large Language Models (LLMs) has substantially facilitated the development of autonomous multi agent systems (MAS). However, the contemporary landscape appears characterized by fragmentation arising from incremental prompt engineering and heterogeneous workflow designs. To mitigate this conceptual disparity, the present survey articulates a unified architectural framework intended to advance MAS research from experimental prototyping toward a systematic architectural science. Recent scholarship is organized along five interacting dimensions comprising Agent Constitution, Multi Agent Orchestration, Evolution, Applications, and Evaluation. Through the decomposition of systems into modular profiles, memory substrates, and interaction topologies, the analysis suggests a discernible transition from heuristic scripts toward executable specifications. Complementing this structural view, application paradigms are synthesized into deterministic Digital Workforces and explorative Digital Laboratories, a dichotomy that implies the necessity for distinct task-oriented and coordination-oriented evaluation methodologies. Looking forward, the study identifies adaptive orchestration, risk governance, and process level observability as likely frontiers for the establishment of dependable multi agent infrastructures.

Abstract content goes here

From Fragmentation to Systematic Design: Architecting LLM-Based Multi-Agent Systems

Caishen Zhou, Yihong Tang, Kehai Chen, Xuefeng Bai, Shuhan Qi, Li Shen, Min Zhang

Abstract—The integration of Large Language Models (LLMs) has substantially facilitated the development of autonomous multi agent systems (MAS). However, the contemporary landscape appears characterized by fragmentation arising from incremental prompt engineering and heterogeneous workflow designs. To mitigate this conceptual disparity, the present survey articulates a unified architectural framework intended to advance MAS research from experimental prototyping toward a systematic architectural science. Recent scholarship is organized along five interacting dimensions comprising Agent Constitution, Multi Agent Orchestration, Evolution, Applications, and Evaluation. Through the decomposition of systems into modular profiles, memory substrates, and interaction topologies, the analysis suggests a discernible transition from heuristic scripts toward executable specifications. Complementing this structural view, application paradigms are synthesized into deterministic Digital Workforces and explorative Digital Laboratories, a dichotomy that implies the necessity for distinct task-oriented and coordination-oriented evaluation methodologies. Looking forward, the study identifies adaptive orchestration, risk governance, and process level observability as likely frontiers for the establishment of dependable multi agent infrastructures.

Index Terms—Large Language Models, Multi-agent systems, Agent Architecture, Multi-Agent Orchestration.

I. INTRODUCTION

A. Motivation

The integration of Large Language Models (LLMs) has facilitated the development of agentic systems capable of following natural language instructions, utilizing tools, and interacting within complex digital environments. Building upon these capabilities, contemporary research has focused on constructing systems composed of multiple LLM-based agents that coordinate through communication and shared memory, a paradigm established by frameworks such as AutoGen [1] and CAMEL [2]. Such multi-agent systems (MAS) are currently being explored across domains ranging from software engineering and data analysis to web automation and social simulation. However, the prevalence of incremental prompt engineering and ad hoc workflow construction has resulted in a rapidly expanding yet conceptually fragmented landscape, complicating system comparison and the accumulation of architectural insight.

This survey posits that LLM-based multi-agent systems are transitioning from experimental prototyping toward a systematic architectural science. Rather than viewing systems as disparate collections of prompts and scripts, MAS are

conceptualized here as structured assemblies of agents, tools, and coordination mechanisms amenable to description and comparison along common dimensions. The focus remains on language-centric systems where LLMs provide the core reasoning capability. Consequently, references to classical multi-agent reinforcement learning or control-oriented MAS are limited to instances where they directly inform current LLM-based architectures.

B. Scope

To elucidate this architectural space, recent scholarship is organized along five interacting dimensions. *Agent Constitution* defines the construction of individual agents, encompassing profiles, memory organization, planning procedures, and tool interfaces. *Multi-Agent Orchestration* delineates how agent collections are arranged into organizational structures, establishing communication channels and collaboration patterns. *Agent Evolution and Learning* addresses system adaptation over time, including mechanisms for supervision and self-improvement that refine policies or protocols. These architectural choices are instantiated in *Multi-Agent System Applications*, which analyze concrete domains such as digital workforces and laboratories with an emphasis on reusable deployment patterns. Finally, *Datasets and Evaluation Systems* examine the environments and metrics that render these designs observable, covering task outcomes, interaction processes, and risk indicators.

Within this framework, concerns regarding role specification, channel design, memory layout, and learning mechanisms are elevated from isolated implementation details to recurring architectural considerations. The objective is not to prescribe a single canonical architecture but to offer a set of dimensions along which existing and future systems can be situated and refined. This architectural lens complements capability-oriented views by foregrounding how design decisions in one dimension may constrain or enable choices in others.

C. Contributions

Recent surveys have reviewed LLM-based agents from various perspectives. While some works provide valuable overviews of task-oriented frameworks [51] or specific capabilities such as communication patterns [52], others focus primarily on social simulations [53]. Although these studies offer essential insights into application domains, they often fragment the engineering view. Distinct from these approaches, the present work articulates a structural framework that centers on the dimensions along which systems are constructed, orchestrated, and evaluated.

C. Zhou, Y. Tang, K. Chen, X. Bai, S. Qi, and M. Zhang are with Harbin Institute of Technology, Shenzhen, China.

L. Shen is with Sun Yat-sen University, China.

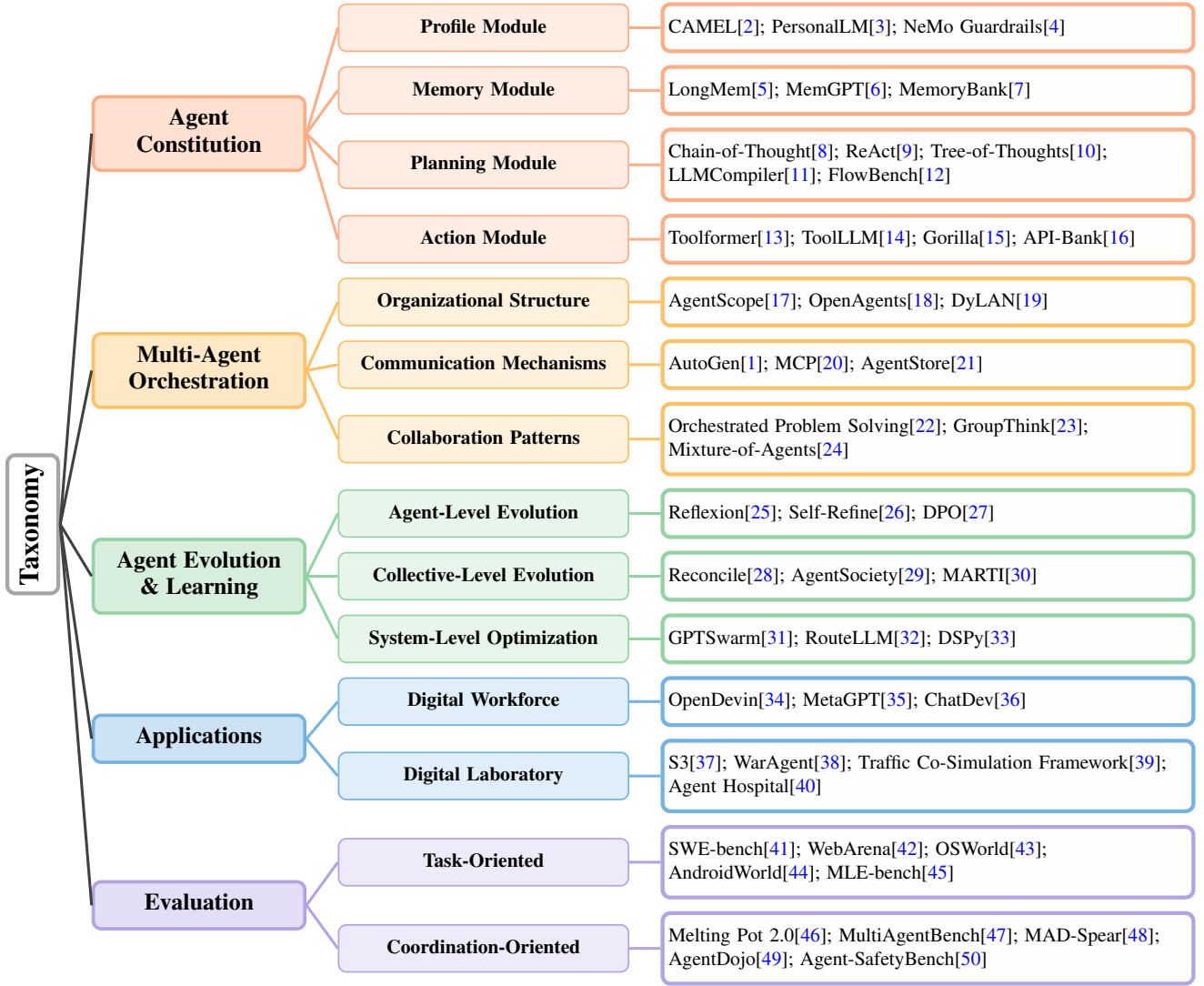


Fig. 1. Architectural taxonomy of LLM-based multi-agent systems.

The contributions of this survey are structured to provide a unified architectural view. The primary contribution involves the articulation of a framework relating agent constitution, orchestration, evolution, application patterns, and evaluation infrastructures. Complementing this theoretical structure, representative systems are synthesized through this multidimensional lens to highlight recurring design patterns, trade-offs, and tensions across domains. Furthermore, the analysis links architectural choices to emerging evaluation practices, emphasizing how process-level and risk-aware metrics enhance the observability and comparability of system behavior. Collectively, these elements aim to clarify key architectural dimensions and offer a structured vantage point for examining subsequent theoretical and empirical work.

II. AGENT CONSTITUTION

The construction of effective multi-agent systems (MAS) relies fundamentally on the architectural design of their constituent agents. Architectural limitations at the individual level, such as instability in role adherence or memory fragmentation,

can impede collective coordination and potentially degrade system-wide performance. To systematize this engineering process, the agent's cognitive architecture is typically decomposed into four interdependent modules. The *Profile* configures agent identity and behavioral scope, while *Memory* facilitates the persistence and retrieval of private and shared context. These substrates enable *Planning* to decompose complex goals under resource constraints, subsequently guiding the *Action* module to execute tasks across tools and peers.

From an engineering perspective, these modules integrate to form a continuous perception–cognition–action loop. Initially, the *Profile* instantiates role parameters and capabilities, providing the static basis for behavior. *Memory* then supplies the dynamic context required for reasoning. Following this, *Planning* determines the sequence of operations, which the *Action* module executes to generate outcomes that are assimilated back into the system state to inform future cycles.

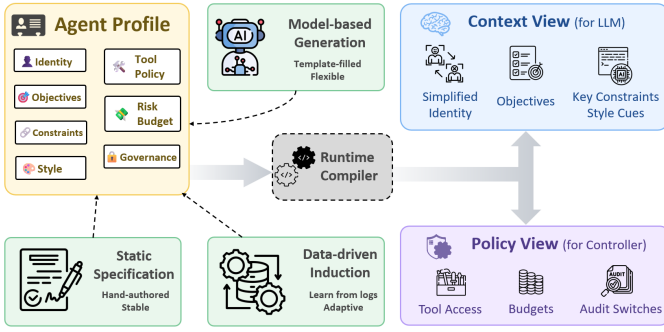


Fig. 2. Profile module: agent profiles instantiated via static, model-based, and data-driven mechanisms, then compiled into context and policy views for execution.

A. Profile Module

The *Profile* module functions as a primary mechanism for agent specialization, transforming a generic LLM into a distinct, modular component within a multi-agent system. Transcending basic prompt-engineered personas, a profile operates as a comprehensive configuration unit that encapsulates identity, functional scope, and operational boundaries. This architectural role is evident across diverse frameworks: pipeline-centric systems, such as *MetaGPT* [35] and *ChatDev* [36], establish static roles for structured production workflows, whereas communicative architectures like *CAMEL* [2] and *AutoGen* [1] enable dynamic role composition and reconfigurable teams. Consequently, within the system architecture, profiles shape not only individual capabilities but also the interaction topology, influencing the exchange of instructions and feedback between roles such as planners and executors.

1) *Static specification*: In regulated or safety-critical settings, static specification typically remains the prevalent strategy. Profiles are manually authored as structured documents that integrate *identity* and *objectives* with operational interfaces, comprising constraints, style, tool policies, and risk posture. Within a multi-agent system, these fields function as coordination primitives, frequently manifesting as typed hand-offs and escalation paths between roles. To promote adherence to such specifications beyond textual prompting, many deployments supplement profiles with execution-time control. This is achieved, for instance, via guardrail frameworks such as NeMo Guardrails [4], constrained decoding models like LlamaGuard [54], or policy templates inspired by Constitutional AI [55] that delimit tool use and memory access. Complementary to runtime guardrails, offline safety alignment techniques based on preference modeling and re-ranking can further harden agent policies prior to deployment [56], [57].

2) *Model-based profile generation*: Complementing manually crafted designs, model-based generation utilizes LLMs to draft, refine, or adapt role descriptions. Techniques originally developed for instruction tuning and alignment [58] and for analysing and optimising system prompts [59] may be repurposed to synthesise role specifications conditioned on task descriptions, organisational structure, or user preferences. This capability proves particularly advantageous in large multi-agent deployments where numerous lightly differentiated roles

are required, or where teams require on-demand reconfiguration. However, automatically induced profiles necessitate validation to mitigate the propagation of bias or misalignment, motivating recent studies on personalised language models and controllable behaviour [3], [60].

3) *Data-driven induction*: Data-driven induction embeds aspects of the profile directly into the model parameters, enabling persistent role specialisation across sessions. Parameter-efficient fine-tuning methods such as LoRA [61] and prompt-tuning [62], [63] allow systems to attach small learned deltas to a structured core profile, achieving personalization while preserving shared interfaces between agents. In multi-agent architectures, this hybrid pattern integrates a stable, declarative role specification with lightweight learned adapters, thereby supporting both consistency at the team level and local adaptability for individual agents.

Beyond these instantiation strategies, operational practice relies on structured representations such as YAML or JSON to facilitate validation and composition across large agent pools, while controllers maintain separate *context views* (providing identity and style cues) and *policy views* (enforcing budgets and access control) [33]. Robust profile design must also account for adversarial and distributional shifts, including indirect prompt injections that potentially subvert role instructions [64]. Collectively, these mechanisms elevate profiles from informal personas to essential architectural artefacts for organising multi-agent systems.

B. Memory Module

The *Memory* module constitutes the architectural substrate through which agents maintain context, accumulate experience, and facilitate coordination. In the construction of multi-agent systems, memory extends beyond single-agent reasoning to mediate the storage, retrieval, and propagation of information across the team. This requirement is typically instantiated through a stratified organisation comprising private memories that encode individual state and shared memories that capture collective knowledge. Recent surveys [65] and interactive studies [66] highlight an emerging shift from ad-hoc caching toward structured designs, where long-horizon behaviours are anchored in consolidated episodic records. Similarly, conversation-programmed frameworks increasingly implement dialogue logs as queryable objects to support coordination and attribution [1].

1) *Private memory architectures*: These integrate volatile working context with persistent long-term stores to support individual agency. Working memory typically operates under finite context limits, frequently employing compression mechanisms such as rolling summaries or token-level pruning, as exemplified by the LLMingua family [67], [68], to retain salient information. For extended contexts, recursive summarisation strategies have been shown to stabilise recall across sessions [69], enabling the accumulation of experience without unbounded context growth. Furthermore, long-term private memory persists observations and reflections into retrievable databases. Systems such as LongMem [70], MemoryBank [7], and MemGPT [6] illustrate how integrating structured storage

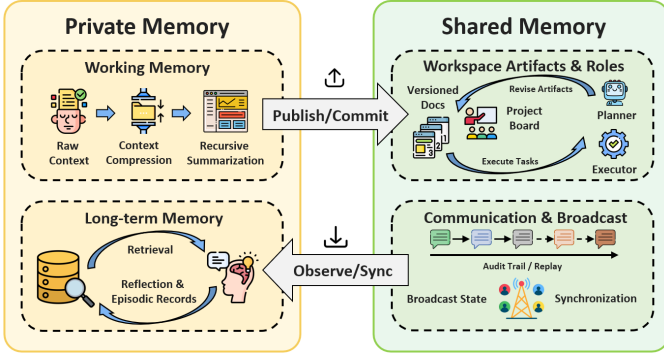


Fig. 3. Memory module: interaction between private memory and shared memory.

with agent controllers can enhance cross-session consistency and personalisation.

2) *Shared memory mechanisms*: From a collaborative engineering perspective, shared memory provides the synchronisation layer for team alignment by externalising intermediate artefacts. Versioned workspaces containing requirements, design documents, or code changes serve as standardised interface boundaries between planner, executor, and critic roles, a pattern adopted by MetaGPT [35] and ChatDev [36]. These shared workspaces act as reference points that reduce ambiguity in multi-stage pipelines. In addition to such artefacts, communication logs record interaction history to facilitate replay and audit [1], while broadcast state mechanisms are employed to maintain consistent world models across larger populations [71]. The architectural configuration of shared memory therefore strongly shapes coordination overhead, credit assignment, and the system’s ability to sustain coherent collective behaviour.

C. Planning Module

The *Planning* module constitutes the architectural engine responsible for decomposing system-level goals into executable sub-tasks and delegating them to appropriate agents. In contrast to single-agent environments where planning prioritizes individual action sequences, multi-agent system construction necessitates extending planning capabilities to instantiate collective task allocation. This architectural shift implies that plan structures should ideally incorporate ownership metadata, resource budgets, and hand-off protocols alongside standard execution steps. Benchmarks such as AgentBench highlight how current planning mechanisms interact with realistic tool calls and budget constraints, indicating a need for more explicit planning architectures [72]. Complementary workflow compilers and evaluation suites, including LLMCompiler [11] and FlowBench [12], illustrate how abstract plans can be rendered into concrete API invocations with integrated precondition checks and audit traces.

1) *Intra-agent planning strategies*: At the single-agent level, three planning patterns are particularly amenable to multi-agent coordination. *Linear decomposition* breaks a goal into an ordered sequence of sub-tasks, a pattern critically analysed in studies on LLM planning limitations [73]. *Exploratory*

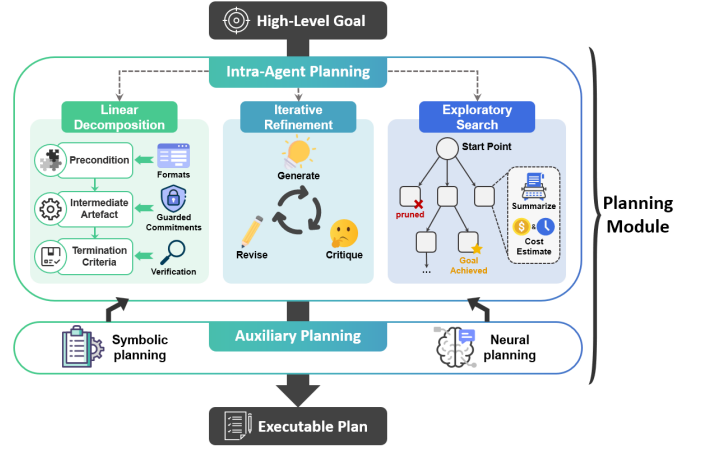


Fig. 4. Planning module: intra-agent planning and auxiliary symbolic/neural planners that turn high-level goals into executable plans.

search methods, such as Tree-of-Thoughts [10] and Language Agent Tree Search [74], expand multiple candidate branches before commitment, trading computational budget for broader solution coverage. *Reflection-driven refinement* integrates iterative critique to revise intermediate plans, as exemplified by Reflexion [25] and Self-Refine [26]. Recent rollback-based correction frameworks further address irreversible error propagation in interleaved think-act trajectories [75]. Within multi-agent architectures, these patterns are lifted from individual reasoning chains to shared plan artefacts: decomposed steps become delegation units, search branches correspond to alternative team strategies, and reflection operates over multi-role execution traces.

2) *Auxiliary planning mechanisms*: Beyond internal reasoning, auxiliary planners augment LLM-based planning with external optimisation. *Symbolic planning* formulations transform tasks into formal specifications such as PDDL or STRIPS, enabling classical planners to satisfy hard constraints and generate machine-checkable plans [73]. In contrast, *neural planning* approaches utilise compact policies or scoring functions to guide next-step selection while the LLM directs abstract reasoning. In multi-agent settings, these learned planners are frequently deployed at the orchestration layer to route sub-tasks based on capability and Service Level Objectives (SLOs). Recent methods such as RouteLLM [32] and Routoo [76] attempt to render such dispatch logic explicit and learnable. Furthermore, tooling such as AutoFlow [77] and WorkflowLlama [78] demonstrates how templates and graph-based execution plans can be compiled from high-level specifications, thereby enhancing the reproducibility and inspectability of multi-agent workflows.

D. Action Module

The *Action* module functions as the execution interface, translating abstract plans into concrete operations within digital or physical environments while mediating interactions with peers. These actions are generally categorized into *environmental* effects that modify the system state and *social* effects that influence other agents. In contemporary architec-

tures, tool use has emerged as a dominant instantiation of this interface, where models call functions, execute code, or exchange structured messages. This pattern is substantiated by corpora and benchmarks such as Toolformer [13], Gorilla [15], ToolLLM [14], API-Bank [16], and StableToolBench [79].

1) *Environmental interaction actions*: These operations interface with external systems by invoking digital tools, executing code, or controlling embodied platforms. For software-facing tasks, LLMs typically issue structured function calls or program fragments to interact with databases, web services, and file systems. Specifications regarding typed tool signatures, which declare arguments, side effects, and idempotency, are commonly paired with JSON-schema-constrained I/O, for example in function-calling leaderboards [80] and real-web evaluation harnesses [81]. In code-centric workflows, sandboxed runtimes with dry runs or staged commits are used to mitigate execution risks during API calls and file edits [82], while repository-grounded evaluations prioritise environment isolation to promote safe dispatch [83]. In embodied settings, natural-language intents are compiled into task or motion plans integrated with safety mechanisms such as collision checks and speed limits. Vision-language-action systems, exemplified by RT-2 [84], DoAsI [85], and OpenVLA [86], illustrate the integration of perception, language, and control within a unified execution loop.

2) *Social interaction actions*: Concurrently, social actions constitute the communication infrastructure between roles within a multi-agent system. Direct exchanges typically utilise structured messages containing machine-readable fields such as intent, evidence, and budget to facilitate task hand-offs, information requests, or artefact critique. Frameworks including *AutoGen* [1] and *AgentVerse* [87] implement typed message APIs and role bindings to support such protocols. Furthermore, specialised runtimes like the Model Context Protocol (MCP) [20] and graph-based orchestrators such as LangGraph [88] help maintain structural consistency across long-horizon dialogues. In contrast to direct messaging, implicit coordination or stigmergy relies on updating shared state, such as writing a claim into a workspace to trigger a review.

E. Summary

The Agent Constitution establishes the architectural foundation of a multi-agent system by transforming stochastic language models into modular, governable components. Through the four interlinked modules of *Profile*, *Memory*, *Planning*, and *Action*, systems define who the agents are, what they know, how they decide, and how they act.

Functioning as the static configuration layer, the *Profile* constrains identity and functional boundaries, while the *Memory* module supplies structured private and shared stores for retaining and synchronising context. On top of these substrates, the *Planning* module decomposes goals into delegable, resource-aware sub-tasks, and the *Action* module realises their effects through tool use, code execution, and social interaction. Across recent systems, a common trend is the shift from informal prompt engineering toward explicit, typed interfaces for roles, state, and execution. This constitutional layer thus provides

the basis upon which later sections organise *orchestration*, *evolution*, and *evaluation* of LLM-based multi-agent systems.

III. MULTI-AGENT ORCHESTRATION

Building upon the foundational agent constitution, this section examines *orchestration* as the central architectural mechanism for constructing multi-agent systems. From a system engineering perspective, orchestration serves as the structural integration layer that instantiates interaction topologies, communication substrates, and collaboration protocols. While robust architectural design facilitates emergent capabilities unattainable by isolated models, structural misalignment at this layer may introduce coordination overhead that compromises system performance, regardless of individual agent competency. Accordingly, the following analysis characterizes the construction of MAS along three primary dimensions: *Organizational Structure*, *Communication Mechanism*, and *Collaboration Pattern*, concluding with an evaluation of design trade-offs and potential failure modes.

A. Organizational Structure

Organizational structure defines the interaction topology and distribution of control logic, thereby determining the architectural placement of planning, execution, and verification modules. As indicated by recent evaluations, structure functions as a foundational architectural decision that fundamentally influences system latency, fault tolerance, scalability, and adaptability [89], [22], [17].

1) *Hierarchical*: Command-control architectures channel control flow and information along a pyramidal chain: upper-tier nodes orchestrate global planning and task decomposition, while lower-tier agents process assigned subtasks and propagate status updates upward. Practical platforms such as AgentScope [17] and OpenAgents [18] demonstrate how centralized control facilitates deterministic behavior and verifiable execution paths. Furthermore, explicit compile-time and runtime policies, specifically those governing resource allocation [90] or system-level verification [91], streamline the implementation of safety-critical operations. However, this centralization may introduce single points of failure and throughput bottlenecks under non-stationary conditions, limitations noted in large-scale deployments [92].

2) *Decentralized*: Peer-to-peer architectures operate without a central coordinator; instead, peers utilize local protocols to interface with neighbours, allowing macroscopic order to emerge from micro-interactions. This topology offers potential robustness against single-node failures, enables natural parallelism, and supports graceful scaling, as evidenced by social-dynamics testbeds like Melting Pot 2.0 [46]. A primary engineering challenge lies in mechanism design, specifically the need to promote convergence and stability under partial observability. Recent frameworks such as GPTSwarm mitigate these coordination complexities by optimizing interaction graphs directly, representing agents as nodes and communication channels as edges within a flat topology [31].

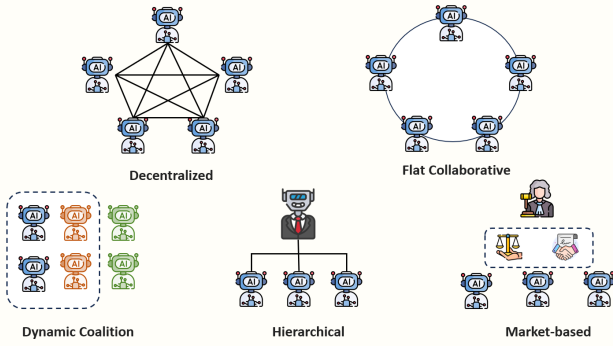


Fig. 5. Organizational structure patterns for multi-agent orchestration.

3) *Flat collaborative*: Consensus-driven teams connect peers in a non-hierarchical topology to pursue shared objectives. Consensus is typically achieved through dialogue, critique, and voting mechanisms rather than directive command. Frameworks such as Orchestrated Problem Solving illustrate the utility of flexible role adoption and rapid knowledge propagation [22]. In parallel reasoning architectures, protocols like GroupThink suggest how concurrent agents collaborating at fine granularity can enhance reasoning quality [23], though such structures may also be susceptible to non-convergence or stalemates in the absence of arbitration logic. Ensemble-style mixtures, exemplified by Mixture-of-Agents, provide a middle ground by structuring critique and aggregation without imposing a rigid control hierarchy [24].

4) *Dynamic coalitions*: Task-centred federations dynamically instantiate ephemeral agent clusters around tasks and dissolve them upon completion, enabling the system architecture to adapt to non-stationary requirements. Systems such as DyLAN [19] and Internet-of-Agents [93] report improved utilisation and time-to-solution via resource-aware composition. However, coordination overhead may arise from coalition discovery and negotiation processes. In practice, lightweight protocols and in-conversation team adaptation serve to mitigate these computational costs [94].

5) *Market-based*: Structures grounded in mechanism design distribute tasks via auction or pricing protocols, where agents bid under constraints and mechanisms determine allocation. Rigorously designed markets can deliver incentive-compatible, cost-aware resource allocation and scale effectively under heterogeneous capabilities, as evidenced by AucArena [95] and studies on LLM-based auctions [96]. Nevertheless, these architectures tend to introduce protocol complexity and potential exposure to strategic manipulation. Recent work highlights specific collusion risks [97] and proposes safeguards against strategic bidding [98].

Architectural selection strategies typically map system uncertainty and task coupling to the corresponding structural topologies. Specifically, tightly constrained, safety-critical workflows often necessitate hierarchical control configurations to guarantee deterministic execution flows. In contrast, non-stationary environments characterized by loose coupling are frequently addressed through decentralized or coalition-based instantiations. To reconcile these competing requirements,

hybrid architectures have emerged as a prevalent construction paradigm within contemporary system designs. This integration is exemplified by architectures that couple centralized control layers with decentralized execution nodes [90], as well as those that layer dynamic coalition mechanisms atop market-based interaction substrates [19], [93].

B. Communication Mechanism

Communication infrastructure serves as the architectural backbone for information synchronisation, intent grounding, and coordination. Consequently, the architectural selection of channel topologies fundamentally influences system throughput, latency, reliability, and security, necessitating architectural alignment with the organisational structure and task demands [99], [100]. Furthermore, recent studies indicate that channel design intrinsically interacts with failure modes such as instability and nondeterminism, underscoring the importance of explicit specifications and observability in system design [101].

1) *Direct channels*: Direct communication typically implements unicast or small-group multicast with explicit addressing. Payloads are structured to range from free-form language to typed messages containing speech-act annotations such as assign, request info, or critique, alongside tool descriptors subject to downstream validation. This architectural pattern facilitates the tight feedback loops necessary for real-time grounding and attribution, a characteristic central to hierarchical frameworks like AutoGen [1] and deliberation protocols [102]. However, scalability constraints may emerge as network complexity increases. To mitigate quadratic growth in message volume, systems increasingly integrate routing optimisation [52] and context pruning mechanisms [103].

2) *Indirect mechanisms*: Indirect architectures architecturally decouple senders from receivers using a shared substrate. Blackboard-style architectures configure agents to publish and consume artefacts, ranging from requirements to executable code, within a central workspace managed by versioning and concurrency control. This approach transforms artefacts into stable coordination primitives, a design central to memory-centric frameworks like AgentStore [21] and repository-based systems like SWE-agent [82]. Alternatively, stigmergic coordination replaces explicit messaging with lightweight state modifications that trigger downstream actions. While this mechanism scales effectively to large swarms, as seen in OASIS [104], it may trade off delivery guarantees for scalability.

3) *Group-level dissemination*: Group-level strategies establish system-wide situational awareness. Broadcast mechanisms support one-to-all announcements, such as simulation ticks or global alerts, to synchronise populations simultaneously. In contrast, publish-subscribe architectures introduce brokered topics to decouple producers from consumers, allowing receivers to filter data by subscription to manage load. This pattern is exemplified by AgentScope [17]. While these mechanisms are well-suited for city or platform-scale contexts, they often require disciplined schema design and back-pressure controls to manage overload, challenges highlighted in large-scale simulations [104].

4) *Protocol-driven*: Protocol-based interaction structures dialogue into formally typed exchanges, including auctions, bargaining, debate, and consensus. In these settings, messages encode bids, proposals, or proofs alongside explicit termination conditions, a structure advocated by the Model Context Protocol [20] and standards for agent-ready APIs [105]. The primary engineering benefits are auditability and the potential for formal analysis of agreement rules. However, multi-party protocols can incur engineering overhead and, without robust architectural safeguards, may remain susceptible to manipulation or collusion under strategic behaviour, as highlighted by adversarial benchmarks [48] and multi-agent security studies [106].

From an implementation standpoint, quality dimensions are typically integrated within the orchestration specification. Key operational attributes include schema alignment, delivery guarantees ranging from at-least-once to exactly-once, and message ordering definitions such as per-sender FIFO versus total ordering. In asynchronous environments, shared substrates further necessitate specified consistency latency and back-pressure mechanisms to handle load spikes. Security provisions, particularly redaction and privacy preservation, function as critical integrity mechanisms [107], while observability via distributed tracing ensures system-wide auditability [108]. For resilience evaluation, standard reporting metrics include throughput and p95 latency under fixed budgets, alongside duplicate delivery rates. Such metrics enable fair comparison across communication architectures, as evidenced by benchmarks for dynamic orchestration [109], [110]. To ensure metric validity, complementary techniques like grammar-constrained decoding maintain structural compliance, preventing parsing failures from confounding latency measurements [111].

C. Collaboration Pattern

Collaboration patterns structure the flow of tasks and information, functioning within the architectural topology defined by the organizational structure and communication substrate. This design space is typically characterized by three distinct dimensions: *collaborative intent*, *collaborative workflow*, and *collaborative protocol*. This modular separation facilitates the independent modeling of goals, execution processes, and interaction specifications before their eventual composition. Such a layered perspective corresponds with recent theoretical frameworks, including RoundTable-style analyses [28] and comprehensive evaluation boards like AgentBoard [112].

1) *Collaborative intent*: Collaborative intent defines the foundational alignment of agent objectives within the system architecture. Under *cooperation*, agents are architected to pursue a shared, positive-sum objective. Research on Teaming Language Agents indicates gains from role-conditioned team formation in complex reasoning tasks [113], while AgentSociety demonstrates that committee-style discussion can improve sample efficiency in social simulations [29]. In contrast, *competition* architectures leverage adversarial dynamics to enhance robustness and reveal exploitability. Studies on rhetorical negotiation [114] and bargaining dynamics [115] quantify strategic manipulation risks within adversarial testbeds. Bridging

these paradigms, *coopetition* structures macro-level alignment with micro-level competition, where platform-wide objectives remain shared while specialised agents compete for selection. This structural tension may yield resilient behaviours, as observed in multi-agent debate frameworks [116] and ensemble architectures like Mixture-of-Agents [24].

2) *Collaborative workflows*: To implement these objectives, collaborative workflows instantiate reusable process templates ranging from linear chains to complex loops. Pipeline architectures typically route artefacts through specialised roles, exemplified by the sequential interactions between planner, executor, and verifier roles common in repository-centric systems like OpenDevin [34] and AgentLite [117]. Similarly, frameworks like TaskWeaver [91] formalise chains comprising retrieval, integration, and quality assurance utilizing typed hand-offs. Extending beyond linear sequencing, parallel aggregation architectures enable independent workers to contribute partial solutions fused via voting or stacking, a pattern where mixture-of-agents ensembles consistently report accuracy gains [24]. Further increasing architectural complexity, iterative feedback loops are employed to refine artefacts until acceptance criteria are met [118], and discursive debate cycles serve to calibrate confidence under resource budgets [119]. In the domain of autonomous improvement, self-play and autocurricula mechanisms generate tasks and critiques to bootstrap skills, reporting transferable competencies in open-ended simulators like Voyager [120] and embodied settings like Eureka [121].

3) *Collaborative protocols*: Structuring the interactions within these workflows, collaborative protocols specify the binding interaction standards for communication. Task allocation is frequently implemented via auction or bargaining protocols utilizing well-typed messages, a structured approach that facilitates auditable allocation and renders strategic behaviour analyzable [122], [35]. In decentralised settings, information diffusion relies on broadcast or topic-based dissemination, where simulators operating at city scales quantify timeliness and coverage under explicit retention policies [99]. To promote system reliability, human-in-the-loop protocols integrate explicit checkpoints for halting, reviewing, and approving actions, supported by provenance snapshots for rollback [112]. When disagreements persist, conflict resolution mechanisms incorporate majority or quorum voting, ballots weighted by competence, or designated arbiters. Comparative studies on committees [119] and conversational dynamics [123] report improved stability and traceable rationales under these mechanisms.

D. Summary

Orchestration constitutes the architectural fabric that integrates disparate agents into a cohesive system. Through the alignment of structure, communication, and workflow via explicit specifications that define resource allocation, protocols, and traceability, multi-agent systems can achieve capabilities surpassing single-agent baselines. To systematize these improvements, controllers and schedulers implement these architectural designs by managing task routing, configuring resource boundaries, and recording provenance.

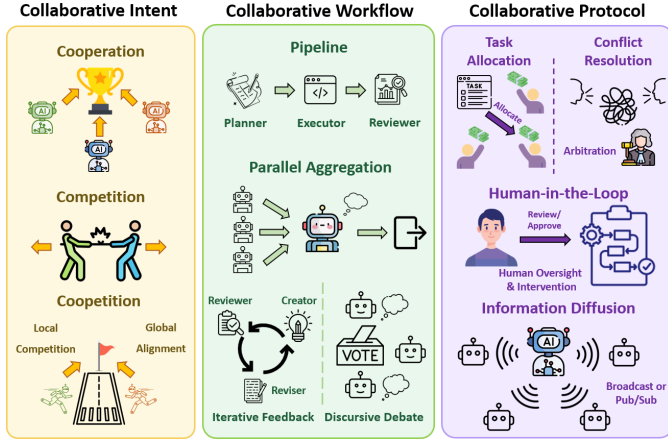


Fig. 6. Collaboration patterns in multi-agent systems.

Building on this foundation, future architectural developments are anticipated to converge on three critical dimensions. Adaptive orchestration mechanisms facilitate the online reconfiguration of structures and routes, balancing stability with cost constraints. Simultaneously, verifiable interaction protocols are expected to integrate safety and liveness checks directly into the runtime execution flow, instantiating auditable hand-offs. Furthermore, cost-aware coordination necessitates the coupling of orchestration logic with serving-layer scheduling, promoting the scalability of system-level Service Level Objectives (SLOs) across teams and domains. Collectively, these directions advance the engineering of scalable and trustworthy multi-agent architectures.

IV. AGENT EVOLUTION AND LEARNING

While *Agent Constitution* and *Multi-Agent Orchestration* establish the static functional baseline of the system, *Evolution and Learning* address the mechanisms by which architectures dynamically reconfigure through experience. From a system construction perspective, this adaptability functions as a critical architectural property, essential for preserving team alignment and minimizing coordination overhead over long horizons. This section examines mutable architectural elements, ranging from profiles and policies to shared memory indices, alongside the feedback vectors that drive their modification.

Evolutionary mechanisms are categorized into three architectural levels based on their scope of impact. *Agent-Level Evolution* operates as the micro-foundation for system robustness, enabling individual agents to self-correct without extensive external intervention. *Collective-Level Evolution* exploits the interaction protocols of the multi-agent system, utilizing peer critique and consensus as mechanisms for mutual improvement. Finally, *System-Level Optimization* targets the orchestration layer itself, dynamically refining routing logic and interaction topologies to maximize global efficiency.

A. Agent-Level Evolution

Agent-level evolution defines the capacity of single agents to update their internal state definitions, specifically profiles,

memory, and action spaces, in response to execution feedback. This mechanism serves as a decentralized error-correction layer, preventing local failures from propagating to the orchestration level.

1) *Reflection as State Update*: Agents analyze execution traces to identify logical flaws or tool misuse, subsequently converting these insights into structured memory entries. Rollback-style correction introduces an explicit backtracking operation that can prevent irreversible error propagation in interleaved think-act trajectories [75]. Frameworks such as Reflexion [25] and Self-Refine [26] exemplify this pattern by externalizing corrective signals into intermediate textual state, conditioning future behavior without parameter updates. In code-centric environments, systems like AutoCodeRover [124] and CodeAct [125] utilize interpreter or test feedback as high-fidelity signals to refine candidate actions and local repair strategies.

2) *Parameter-Efficient Policy Optimization*: Beyond transient state updates, agents require persistent improvements to their core policy models. While traditional Reinforcement Learning from Human Feedback (RLHF) aligns models via reward modeling, recent advances focus on direct preference optimization to reduce training overhead. Algorithms such as DPO [27] and SimPO [126] allow agents to internalize feedback from humans or environmental verifiers directly into their weight representations, serving as the computational engine for persistent skill acquisition. Recent work further explores compute-efficient safety alignment via preference re-ranking and lightweight reward modeling [56], while broader surveys systematize the design space of human preference learning for aligning LLM policies [57].

3) *Search-Based Deliberation*: To preempt errors before execution commitments, agents employ look-ahead search strategies. Techniques such as Tree-of-Thoughts [10] and Language Agent Tree Search [74] allow agents to simulate future states and prune infeasible paths. Within a multi-agent architecture, these deliberation processes can be parallelized across peer nodes, effectively trading computational budget for enhanced individual reliability.

B. Collective-Level Evolution

Moving beyond individual correction, *collective-level evolution* exploits the heterogeneous perspectives within a multi-agent system to drive mutual adaptation. This process operates across scales: within local teams, interaction protocols function as the learning substrate where peer critique and consensus serve as supervision signals; across larger populations, co-evolutionary dynamics leverage diversity to sustain continuous improvement without expensive human annotation.

1) *Consensus and Discursive Debate*: Multi-agent systems uniquely enable the refinement of outputs through discursive interaction. Mechanisms such as multi-agent debate [116] and round-table negotiation [28] encourage agents to justify their reasoning against peer scrutiny, a process that can expose logical fallacies. Recent analyses suggest that while multi-agent discussions often improve reasoning over single-agent baselines, the gains are sensitive to aggregation protocols and may

require arbitration to mitigate groupthink [119]. Furthermore, large-scale simulations like AgentSociety provide platforms to study how local interactions influence the emergence of social norms, offering a testbed for understanding policy dynamics in synthetic populations [29].

2) *Knowledge Distillation and Skill Transfer*: To consolidate collective insights into persistent capabilities, architectures increasingly explore cross-agent distillation. Frameworks like Mixture-of-Agents demonstrate that aggregating diverse heuristics from multiple models at inference time can substantially improve task performance [24], suggesting a potential pathway for distilling ensemble intelligence into smaller models. Similarly, adaptive teaming strategies allow agents to learn from role-conditioned partners during execution. By observing successful collaboration patterns, agents can internalize superior task decomposition strategies, as reported in studies on teaming language agents [113]. This concept aligns with emerging frameworks like AIME, which introduce dynamic planning and agent instantiation mechanisms that could support the evolution of interoperation rules [127].

3) *Population-Based Co-Evolution via LLM-MARL*: At the macro scale, population-based strategies integrate principles from Multi-Agent Reinforcement Learning (MARL) to drive continuous improvement. By dynamically pairing agents with evolving opponents or partners, systems create an autocurriculum where task difficulty scales with population capability. This dynamic is essential for preventing strategy collapse in open-ended environments, and is instantiated in adversarial arenas such as AucArena [95], which probe strategic planning under auction dynamics. Complementarily, self-play fine-tuning methods like SPIN [128] demonstrate how iterative self-play can strengthen single models without external labels, offering a template for population-style improvement. Recent frameworks like MARTI further formalize this by integrating centralized interaction with distributed policy training, showing that RL-trained multi-agent systems can outperform comparable single-agent baselines under fixed inference budgets [30]. For a broader perspective on reinforcement fine-tuning in multi-agent settings, we refer to emerging surveys that explicitly connect MARL with LLM-based systems [129], [130].

C. System-Level Optimization

At the macro level, evolution targets the orchestrator itself, shifting the focus from optimizing individual agents to refining the structural and routing parameters that govern system-wide coordination.

1) *Dynamic Topology and Protocol Synthesis*: Advanced architectures treat the interaction graph as a learnable object rather than a static artifact. Frameworks such as GPTSwarm [31] model the multi-agent system as an optimizable graph, where automatic graph optimizers adjust connectivity, for instance by adding or pruning edges, to maximize task performance. Complementing this structural adaptation, automated prompt optimization strategies apply iterative search procedures to refine global instructions. Methodologies like OPRO [131] and APO [132] treat natural-language

instructions as a search space; in a multi-agent context, these techniques can be applied to optimize role definitions and hand-off protocols without human intervention.

2) *Learned Routing and Resource Allocation*: To balance cost and quality, the routing logic itself undergoes optimization via data-driven learning. Methods such as RouteLLM [32] train dispatchers using preference data, enabling the system to route queries to the most cost-effective model based on complexity. Similarly, cascading architectures exemplified by FrugalGPT [133] dynamically compose multiple models, escalating to more capable but expensive LLMs only when necessary. To operationalize these updates, declarative frameworks like DSPy [33] compile high-level intent into optimized execution graphs, establishing a rigorous pipeline where architectural changes remain traceable and auditable.

D. Summary and Outlook

Agent evolution bridges the gap between static capabilities and dynamic environments by enabling architectural reconfiguration at the individual, collective, and system levels. By integrating agent-level self-correction, collective-level evolution, and system-level optimization, MAS architectures can accumulate competence and efficiency over time.

Future research must prioritize the stability of these learning loops. Critical frontiers include developing verifiable self-improvement protocols to prevent reward hacking and ensuring that distributed updates do not destabilize system-wide coordination specifications. Furthermore, the integration of lifelong learning mechanisms remains essential to manage the accumulation of knowledge without suffering from catastrophic forgetting or context overflow.

V. MULTI-AGENT SYSTEM APPLICATIONS

Having established the three core construction dimensions of *Agent Constitution*, *Multi-Agent Orchestration*, and *Agent Evolution*, this section examines how these abstract components are instantiated and assembled to address real-world problems. In engineering practice, there is no single “optimal” multi-agent architecture; instead, the specific configuration of agents and their interaction topology is dictated by the operational goals of the domain.

This architectural variability is examined through two primary paradigms that instantiate the dimensions of *Agent Constitution*, *Multi-Agent Orchestration*, and *Agent Evolution* in distinct configurations. The *Digital Workforce* typically characterizes agents as deterministic executors, employing hierarchical orchestration and static profiles to prioritize reliability and efficiency in domains such as software engineering and finance. Conversely, the *Digital Laboratory* predominantly establishes agents as autonomous entities, utilizing decentralized interaction and evolutionary memory to simulate complex social dynamics or economic systems. To systematize these architectural distinctions, Table I serves as a structural taxonomy, cross-referencing representative frameworks with the core dimensions established in the preceding sections to provide an organizational framework for the domain-specific discussions that follow.

A. Applications as a Digital Workforce

In the engineering of a Digital Workforce, architectural design typically emphasizes reliability, cost-efficiency, and auditability over emergent behaviors. This operational imperative motivates specific structural choices: the instantiation of static, expert-level *Profiles* to establish functional specialization; the adoption of hierarchical or market-based *Orchestration* to facilitate predictable execution; and the delimitation of *Evolution* to convergence-oriented mechanisms such as error correction. Production deployments illustrate how these abstract components are composed to deliver tangible outcomes across diverse industries.

1) *Intelligent manufacturing*: In flexible job-shop settings, the architecture typically pairs local decision-makers, such as machine or robot agents instantiated with narrow, equipment-specific profiles, with global coordinators to optimize makespan and utilisation. Orchestration logic is frequently instantiated via graph-aware reinforcement learning schedulers to promote robustness against machine failures and to adapt dispatch policies from logged production data [134]. Alternatively, dynamic dispatching mechanisms may utilize auction protocols or coalition formation to handle rush orders, effectively decentralizing the control plane [135]. Recent designs further integrate LLM-mediated planners with shop-floor controllers. In these systems, manager agents decompose orders against standard operating procedures before dispatching tasks to logistics controllers, a pattern that prioritizes the synchronization of digital twins with physical states [136], [137].

2) *Supply-chain optimisation*: To integrate distributed information sources across procurement and logistics, architectures in this domain depend fundamentally on shared *Memory* modules. A prevalent design pattern combines blackboard-style event stores for tracking delays with publish-subscribe communication topics for inventory pricing. This configuration facilitates asynchronous coordination without necessitating synchronous meetings, as detailed in autonomous logistics frameworks [138] and coordination studies [139]. Regarding control logic, systems often instantiate multi-agent reinforcement learning to optimize replenishment strategies under stochastic lead times [140]. From an evaluation perspective, simulators facilitate counterfactual stress testing prior to rollout [141], while recent work on federated multi-agent reinforcement learning frames cross-firm information sharing as a governed resource, subject to confidentiality constraints and access policies [142].

3) *Financial services and trading*: Architectures in finance typically prioritize risk management through the composition of heterogeneous agent ensembles. Analyst, critic, and trader roles are instantiated to fuse fundamental, quantitative, and sentiment signals at decision time. Collaboration patterns frequently employ debate-oriented protocols to synthesise these views before committing positions, an approach validated in FinCon [143] and broader trading agent architectures [144]. In parallel, multi-agent reinforcement learning populations are trained to discover complementary strategies for market making. These populations operate under strict inventory and

impact constraints to yield risk-adjusted gains, as demonstrated by algorithmic trading baselines [145]. Moreover, verification layers integrate approval gates and provenance tracking for language-driven agents, promoting regulatory compliance before live deployment [146].

4) *End-to-end knowledge work*: In software engineering, role-first orchestrations manage the complete lifecycle through the rigorous definition of *Profiles* and *Workflows*, typically under a hierarchical orchestration structure. Frameworks such as MetaGPT [35] and ChatDev [36] explicitly configure responsibilities and escalation paths for product managers, architects, and engineers, while AutoGen provides the underlying conversational substrate [1]. These multi-agent workforces increasingly target complex software and MLOps pipelines, with repository-level issue resolution formalised as benchmark tasks in SWE-bench [41] and end-to-end machine learning engineering workflows captured by MLE-bench [45]. To handle such intricacy, execution-centric agents integrate retrieval with patch synthesis and critique, often utilizing specialized Engineer and Reviewer roles as instantiated in systems like OpenDevin [34] and AutoCodeRover [124]. Consequently, evaluation metrics increasingly probe multi-file reasoning and realistic repository-level workflows, utilizing diagnostic suites such as DevBench [147] and RepoBench [148].

5) *Data analysis and business intelligence*: Systems in this domain are designed to coordinate retrieval, visual reasoning, and narrative generation under tight latency and cost constraints. Teams operate over shared artefacts, typically managed by declarative controllers. Advanced workflows operationalize a cycle of proposal generation, tool execution, and provenance-aware critique, as seen in InsightPilot [149]. Specialized architectures pair role-based parsing with verification to handle complex chart data, a capability tested by benchmarks like ChartBench [150]. Furthermore, text-to-SQL controllers and multi-agent report builders, exemplified by AgentBoard, incorporate negotiation and repair loops to ensure schema grounding. These systems are constructed to deliver faithful narratives and high-impact decisions while optimizing latency and cost per dashboard [112].

B. Applications as a Digital Laboratory

Constructing a Digital Laboratory requires architects to prioritize behavioral fidelity, emergent complexity, and causal consistency. Unlike the Digital Workforce, where convergence is the goal, this paradigm instantiates agents as autonomous entities to explore open-ended dynamics. This operational objective typically dictates specific design choices: the configuration of expressive *Profiles* with long-term *Memory* to simulate human-like variability; the utilization of decentralized *Orchestration* to allow macro-regularities to emerge from micro-interactions; and the frequent deployment of population-based *Evolution* to discover novel strategies.

1) *Social and behavioural sciences*: To study norm emergence and information diffusion, architectures in this domain typically favour decentralized topologies. By endowing agents with rich profiles and memory streams, these systems generate human-like routines where indirect, environment-mediated

TABLE I
REPRESENTATIVE LLM-BASED MULTI-AGENT APPLICATIONS AS DIGITAL WORKFORCE AND DIGITAL LABORATORY.

Category	Domain	Example	Profile	Topology	Evolution
Digital Workforce	Intelligent Manufacturing	Shopfloor Assembly [136]	Static	Hierarchical	System-level
	Supply-chain opt.	MABIM [141]	Static	Decentralized	System-level
	Finance & Trading	FinCon [143]	Model-based	Hierarchical	Agent-level
	Digital Company	MetaGPT [35]	Static	Hierarchical	System-level
	Data Analysis & BI	Insight Agents [149]	Model-based	Hierarchical	System-level
Digital Laboratory	Social Behaviour	S3 [37]	Data-induced	Decentralized	Collective-level
	System opt. & Digital Twins	LLM-DT Parametrization [137]	Model-based	Hierarchical	System-level
	Game Theory & Strategy	Richelieu [114]	Static	Decentralized	Agent-level
	Human-Agent Sandboxes	Agent Hospital [40]	Model-based	Hierarchical	Agent-level

communication, as prototyped in environments like Melting Pot 2.0 [46], emerges at scale. This stigmergic coordination is observable in large-scale social simulations [92]. Generative communities extend this by reproducing everyday behaviors without hand-coded macro rules, scaling to support diverse interactions as exemplified by the S3 simulation [37] and group collaboration frameworks [151]. Expanding to macro-systems, sandboxes explore complex domain dynamics. WarAgent investigates geopolitical escalation and alliance formation [38], while frameworks such as EconAgent simulate macroeconomic activities to forecast fiscal policy impacts [152]. Given the explanatory nature of these applications, evaluation often prioritizes face validity against stylised facts and causal consistency under exogenous interventions.

2) *System optimisation and digital twins*: In silico MAS environments are constructed to optimise cyber-physical-social systems (CPSS) by enabling counterfactual experiments prior to deployment. Architectures in this domain often integrate global broadcast states for synchronization with local controllers or protocol-driven markets for resource allocation. In the domain of urban mobility, ChatSUMO leverages LLMs to automate traffic scenario generation [153], while co-simulation frameworks integrate infrastructure sensing with reinforcement learning for robust traffic management [39]. In manufacturing settings, multi-agent systems facilitate simulation model parametrization within digital twins, automating the configuration of complex environments [137].

3) *Game theory and strategy discovery*: Multi-agent architectures extend classical game theory by enabling equilibrium finding in regimes of incomplete information and language-mediated coordination. Benchmarks such as AgentBench [72] and AgentBoard [112] explicitly measure these capabilities against strong baselines. To prevent strategy collapse, system designers employ population-based self-play and league training to enforce diversity, particularly when augmented by curricula that vary opponent pools [46]. Beyond abstract

payoffs, rich environments probe distinct cognitive faculties. Diplomacy games isolate negotiation dynamics [114], whereas social-deduction scenarios like AvalonBench test theory-of-mind and deception [154]. Additionally, auction-based mechanisms, as explored in AucArena [95], probe resource allocation under strategic behavior. Consequently, assessment metrics can include empirical win rates alongside theoretical indicators such as exploitability, regret, and Pareto efficiency [72].

4) *Human-agent training sandboxes*: These environments are engineered to facilitate safe, repeatable instruction in domains where real-world practice is prohibitive due to cost or risk. To bound risk while preserving fidelity, systems employ human-in-the-loop protocols that interleave scripted events with explicit gates for halting, reviewing, and approving actions [155], [156]. In emergency response, simulators populate multi-role environments comprising dispatch, rescue, and logistics units to facilitate rehearsal under stress, as demonstrated by Sim911 [157]. Clinical training has scaled to hospital-level simulacra that simulate evolvable medical agents, exemplified by Agent Hospital [40]. Similarly, benchmarks like AgentClinic evaluate multimodal agents in simulated clinical environments to assess diagnostic planning and tool use [158]. Furthermore, XR operating-room settings assign roles such as surgeon and nurse to provide realistic constraints, a capability validated in the SurgBox sandbox [159]. From an architectural standpoint, these sandboxes instantiate rich *Profiles* and long-horizon *Memory*, combined with tightly controlled *Orchestration* and human-in-the-loop *Evolution*, to balance realism against safety.

C. Summary and Outlook

Multi-agent system applications are characterized by two distinct architectural paradigms, each instantiating the core components of *Constitution*, *Orchestration*, and *Evolution* to address specific operational objectives. The *Digital Workforce*

constructs systems for deterministic automation, typically prioritizing reliability and cost-efficiency through hierarchical or market-based orchestration and static profile definitions. In contrast, the *Digital Laboratory* architects environments for exploration and simulation, frequently leveraging decentralized topologies and evolutionary memory to facilitate the emergence of complex macro-behaviors.

Advancing these systems from experimental prototypes to dependable infrastructure requires bridging these regimes. A key architectural frontier is principled transfer: converting behavioral insights and policies from Digital Laboratory simulations into the control specifications required by the Digital Workforce. Operational data from production deployments can then be fed back into experimental sandboxes for continual refinement. This interoperability calls for systematic design frameworks that map architectural choices to application requirements, together with evaluation protocols that rigorously assess behavioral fidelity, safety, and trade-offs among cost, quality, and latency across domains.

VI. EVALUATION

Advancements in multi-agent systems rely fundamentally on credible evaluation infrastructures capable of characterizing the operational resilience of designs spanning *Agent Constitution*, *Multi-Agent Orchestration*, and *Agent Evolution*. Contemporary evaluation practices have diverged into two complementary paradigms, reflecting the dual nature of MAS as both a problem-solving utility and a coordination substrate.

In *task-oriented* settings, the multi-agent team functions as an enhanced problem-solving unit, typically deployed on benchmarks originally conceptualized for single agents. The primary objective in this context is to determine the extent to which mechanisms such as role specialization, debate, or tool-based collaboration enhance downstream task performance. Conversely, *coordination-oriented* settings position the multi-agent system itself as the object of inquiry, utilizing sandbox environments and game-theoretic scenarios to investigate communication efficiency, norm emergence, and strategic behavior. Across these paradigms, rigorous progress necessitates not only diverse environments and datasets but also execution runners and metrics that quantify process dynamics, safety margins, and resource trade-offs, thereby extending assessment beyond outcome success alone.

A. Landscape of Evaluation Paradigms

Current evaluation practices in multi-agent systems generally adhere to two dominant paradigms, reflecting the dual nature of MAS as both a problem-solving utility and a coordination substrate. Recently, emerging methodologies have begun to synthesize these perspectives into dedicated multi-agent benchmarks. To systematize this landscape, Table II classifies representative benchmarks according to their domain and evaluative focus. It explicitly demarcates the architectural boundary between task-oriented environments, which prioritize execution utility, and coordination-oriented sandboxes, which emphasize interaction dynamics.

1) *Task-oriented evaluation*: The predominant approach positions a multi-agent system as an enhanced problem solver, deploying it on benchmarks originally conceptualized for single agents. In this context, the primary objective is to determine whether mechanisms such as role specialization, debate, or tool-based collaboration can surpass the performance of well-engineered single agents on standard tasks. Representative examples include software engineering suites such as SWE-bench [41], which formalise repository-level issue resolution; real-world web interaction environments like WebLINX [160]; OS-level evaluation frameworks such as OS-World [43]; and mobile interaction platforms exemplified by AndroidWorld [44]. In these settings, the evaluation protocol is largely inherited from the underlying single-agent benchmark, focusing on task success, trajectory correctness, and occasionally tool-use fidelity, while the internal collaboration patterns of the team frequently remain opaque.

2) *Coordination-oriented evaluation*: Conversely, a second line of research shifts focus from task completion to the internal dynamics of interaction, utilizing bespoke sandboxes or game-theoretic scenarios to investigate coordination, negotiation, and competition. These interaction-centric substrates typically prioritize repeated-play environments over fixed answer keys, focusing on equilibrium selection, convention formation, and robustness under perturbations. Prominent examples include social dilemma suites such as Melting Pot 2.0 [46], auction-based arenas like AucArena [95] for studying strategic behaviour under market mechanisms, and social-deduction environments such as AvalonBench [154] that assess theory-of-mind and deception capabilities. In these benchmarks, the dataset is defined by initial profiles, game rules, and payoff matrices, with evaluation emphasizing coordination efficiency, stability, and emergent collective behaviour rather than one-shot accuracy.

3) *Unified multi-agent benchmarks*: Bridging these distinct paradigms, recent work aims to design benchmarks that explicitly position the multi-agent system as the primary object of study. MultiAgentBench [47] represents one of the few suites to date that systematically evaluates the collaboration and competition of LLM-based agents across diverse domains. By jointly varying task types, interaction topologies, and coordination protocols, such as group discussion and cognitive planning, it provides milestone-based key performance indicators that capture both task completion and the quality of interaction. This unified design offers a potential template for MAS-specific benchmark families that extend beyond reused single-agent tasks or isolated game environments, providing an integrated suite for the systematic assessment of multi-agent interaction capabilities.

B. Core Evaluation Dimensions

Across task-oriented and coordination-oriented paradigms, existing studies predominantly report metrics derived from single-agent evaluation, often directing limited attention to the interaction-specific properties of multi-agent systems. To advance systematic assessment, emerging practices may be categorized into three complementary dimensions comprising

TABLE II
EVALUATION DOMAINS AND BENCHMARKS FOR LLM-BASED MULTI-AGENT SYSTEMS.

Domain	Representative Benchmarks	Task	Coordination
Code Repositories	SWE-bench [83]	✓	✗
Web / Browser Interaction	WebLINX [160]	✓	✗
OS-level Environments	OSWorld [43]	✓	✗
Mobile Interaction	AndroidWorld [44]	✓	✗
Social Dilemmas	Melting Pot 2.0 [46]	✗	✓
Auctions / Markets	AucArena [95]	✗	✓
Social Deduction	AvalonBench [154]	✗	✓
Mixed Multi-domain Suites	MultiAgentBench [47]	✓	✓

efficacy, behaviour, and risk. These dimensions offer a structural lens for comparing and designing evaluation protocols.

1) *Efficacy*: This dimension concerns whether a multi-agent system completes tasks and the associated computational costs. Core metrics include success rate, solution quality (human or rubric-based), and throughput. Benchmarks such as MultiAgentBench report these measures across collaborative and competitive settings, contrasting single-agent and multi-agent systems on a shared task suite [47]. In code-centric domains, frameworks such as CodeAgents use token- and call-level accounting to quantify orchestration costs, reporting prompt and tool-invocation statistics alongside accuracy [161]. Recent systems also track communication overhead (e.g., message counts and byte volume) to identify regimes where gains are outweighed by coordination costs [52].

2) *Behavioural evaluation*: Beyond end-task outcomes, multi-agent evaluation increasingly seeks to characterize the decision-making process. Behavioural metrics examine the structure, efficiency, and robustness of collaboration. Relevant indicators include role consistency, contribution balance, and the frequency of redundancy or cyclical dialogue. For instance, MultiAgentBench complements task success with indicators of collaboration quality, analyzing coordination success rates and conflict resolution patterns [47]. Communication-centric surveys further identify candidate signals such as dialogue structure and turn-taking regularity [52]. However, reusable behavioural indicators remain relatively under-specified, presenting an opportunity for future benchmark design.

3) *Risk and reliability*: Evaluation frameworks should ideally address system behaviour regarding adversarial inputs, specification gaps, and distribution shifts. Recent analyses characterize failure modes in LLM-based multi-agent systems, identifying specification errors, misaligned roles, and verification gaps, thereby advocating for process-aware risk assessment [162]. Although safety benchmarks increasingly incorporate multi-agent scenarios, quantitative risk metrics, including cascade depth and policy violation rates, remain underrepresented in public leaderboards. Consequently, integrating these diagnostics into general-purpose evaluation harnesses constitutes a persistent design challenge.

C. Summary and Outlook

Comprehensive evaluation of multi-agent systems necessitates transcending binary outcome metrics to encompass the intricate dynamics of interaction, resource consumption, and safety. Current methodologies predominantly bifurcate into task-oriented paradigms, which assess problem-solving utility against single-agent baselines, and coordination-oriented frameworks, which investigate the emergent social dynamics of the system. However, the persistence of protocols derived from single-agent evaluation often obscures the specific architectural costs and behavioral pathologies inherent to distributed collaboration.

Looking ahead, the advancement of evaluation infrastructures for LLM-based multi-agent systems necessitates integrated benchmark designs that jointly vary task types, interaction topologies, and coordination protocols. Such designs should ideally expose interaction processes rather than relying solely on final outcomes. A primary research frontier involves defining reusable behavioural and risk-oriented metrics, specifically standardized descriptors of coordination quality, communication overhead, and the depth or frequency of error cascades and policy violations. These indicators require systematic reporting across diverse environments to ensure comparability. Simultaneously, there is a critical need for datasets and harnesses that align process-level signals with established success measures, enabling principled comparisons between single and multi-agent architectures under shared conditions. Progress in these directions may provide a robust empirical foundation for assessing organizational designs, guiding the development of safety mechanisms, and substantiating claims regarding the capabilities and limitations of multi-agent systems.

VII. CONCLUSION

This survey articulates a systematic architectural perspective for LLM-based multi-agent systems, positioning current practices not merely as collections of heuristic scripts but as a nascent engineering discipline increasingly defined by executable specifications. By synthesizing five core dimensions comprising *Constitution*, *Orchestration*, *Evolution*, *Ap-*

plications, and *Evaluation*, the literature is organized into a unified framework. Within this structure, agent behaviors are instantiated via modular profiles, structured memory, and explicit interaction protocols. This holistic view suggests that the transition from single-model prompting to multi-agent collaboration necessitates a fundamental shift, prioritizing architectural integration and process observability alongside outcome efficacy.

Looking forward, the maturation of multi-agent systems will likely hinge on progress across several strategic frontiers.

1) *Adaptive orchestration and communication efficiency*: Recent scholarship has begun to treat interaction graphs and communication channels as optimizable parameters rather than static design choices. Building upon these developments, future inquiry may investigate meta-controllers capable of adapting topologies, routing policies, and communication budgets online. Such mechanisms would enable collaboration structures to respond dynamically to task uncertainty and resource constraints instead of remaining statically configured.

2) *Safety, robustness, and risk governance*: As deployment expands into high-stakes environments, collective failure modes, including collusion, prompt injection, and cascading specification drift, emerge as critical concerns. A significant avenue for research involves integrating topology-aware safety analyses, adversarial benchmarks, and lightweight runtime monitors into coherent risk-management pipelines that span profiles, memory, and interaction protocols. Complementarily, viewing safety and robustness as an explicit feedback pipeline (internal, external, multi-agent, and human feedback) helps clarify where monitoring signals originate and how they should be operationalized in agent learning loops [163].

3) *Unified Benchmarking*: Current evaluation practices for LLM-based multi-agent systems are characterized by fragmentation, where framework-specific definitions of tasks, logging formats, and success criteria impede architectural comparisons. This lack of standardization often reduces assessment to singular outcome scores, obscuring the internal dynamics of collaboration. Although emerging benchmarks increasingly quantify dimensions such as robustness and rationality, they frequently lack a shared, process-aware trace schema. Consequently, a unified evaluation standard necessitates extending analysis beyond task success to elucidate the execution process, specifically capturing communication patterns, tool usage, failure modes, and the trade-offs between cost and latency. Aligning these process-level standards with portable specifications for agent roles and memory states appears essential for transforming ad-hoc orchestrations into an interoperable and cumulative engineering discipline.

Ultimately, the utility of multi-agent systems will likely be determined by the rigor of their construction. By integrating expressive architectures with rigorous observability and economic rationality, the field may transition from experimental prototyping toward the establishment of dependable, mission-critical infrastructures, with this survey serving as an architectural scaffold for such a transition.

REFERENCES

- [1] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu *et al.*, “Autogen: Enabling next-gen llm applications via multi-agent conversation,” in *First Conference on Language Modeling*, 2024.
- [2] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, “Camel: Communicative agents for “mind” exploration of large language model society,” *arXiv preprint arXiv:2303.17760*, 2023.
- [3] H. Jiang, X. Zhang, X. Cao, C. Breazeal, D. Roy, and J. Kabbara, “Personallm: Investigating the ability of large language models to express personality traits,” in *Findings Assoc. Comput. Linguistics: NAACL 2024*, Mexico City, Mexico, 2024, pp. 3605–3627.
- [4] T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen, “Nemo guardrails: A toolkit for controllable and safe llm applications with programmable rails,” *arXiv preprint arXiv:2310.10501*, 2023.
- [5] D. Wu, H. Wang, W. Yu, Y. Zhang, K.-W. Chang, and D. Yu, “Longmemeval: Benchmarking chat assistants on long-term interactive memory,” *arXiv preprint arXiv:2410.10813*, 2024.
- [6] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica *et al.*, “Memgpt: Towards llms as operating systems,” 2023.
- [7] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang, “Memorybank: Enhancing large language models with long-term memory,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 17, 2024, pp. 19 724–19 731.
- [8] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24 824–24 837.
- [9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan *et al.*, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations*, 2023.
- [10] S. Yao, D. Yu, J. Zhao, I. Shafraan, T. L. Griffiths, Y. Cao *et al.*, “Tree of thoughts: Deliberate problem solving with large language models,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 52 314–52 336.
- [11] C. Cummins, V. Seeker, D. Grubisic, B. Roziere, J. Gehring, G. Synnaeve *et al.*, “Meta large language model compiler: Foundation models of compiler optimization,” in *Proceedings of the 33rd ACM SIGPLAN International Conference on Compiler Construction*, 2025, pp. 119–131.
- [12] R. Xiao, W. Ma, K. Wang, Y. Wu, J. Zhao, H. Wang *et al.*, “Flowbench: Revisiting and benchmarking workflow-guided planning for llm-based agents,” *arXiv preprint arXiv:2406.14884*, 2024.
- [13] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, L. Zettlemoyer *et al.*, “Toolformer: Language models can teach themselves to use tools,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 68 539–68 555.
- [14] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu *et al.*, “Toolllm: Facilitating large language models to master 16000+ real-world apis,” *arXiv preprint arXiv:2307.16789*, 2023.
- [15] S. G. Patil, T. Li, T. Zhang, X. Li, Y. S. Li, I. Stoica *et al.*, “Gorilla: Large language model connected with massive APIs,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 11 158–11 173.
- [16] M. Li, Y. Zhao, B. Yu, F. Song, H. Li, H. Yu *et al.*, “Api-bank: A comprehensive benchmark for tool-augmented llms,” *arXiv preprint arXiv:2304.08244*, 2023.
- [17] D. Gao, Z. Li, X. Pan, W. Kuang, Z. Ma, B. Qian *et al.*, “Agentscope: A flexible yet robust multi-agent platform,” *arXiv preprint arXiv:2402.14034*, 2024.
- [18] T. Xie, F. Zhou, Z. Cheng, P. Shi, L. Weng, Y. Liu *et al.*, “Openagents: An open platform for language agents in the wild,” *arXiv preprint arXiv:2310.10634*, 2023.
- [19] Z. Liu, Y. Zhang, P. Li, Y. Liu, and D. Yang, “A dynamic llm-powered agent network for task-oriented agent collaboration,” *arXiv preprint arXiv:2310.02170*, 2024.
- [20] X. Hou, Y. Zhao, S. Wang, and H. Wang, “Model context protocol (mcp): Landscape, security threats, and future research directions,” *arXiv preprint arXiv:2503.23278*, 2025.
- [21] C. Jia, M. Luo, Z. Dang, Q. Sun, F. Xu, J. Hu *et al.*, “Agentstore: Scalable integration of heterogeneous agents as specialized generalist computer assistant,” *arXiv preprint arXiv:2410.18603*, 2024.
- [22] S. Rasal and E. J. Hauer, “Navigating complexity: Orchestrated problem solving with multi-agent llms,” *arXiv preprint arXiv:2402.16713*, 2024.

- [23] C.-J. Hsu, D. Buffelli, J. McGowan, F.-T. Liao, Y.-C. Chen, S. Vakili *et al.*, “Group think: Multiple concurrent reasoning agents collaborating at token level granularity,” *arXiv preprint arXiv:2505.11107*, 2025.
- [24] J. Wang, J. Wang, B. Athiwaratkun, C. Zhang, and J. Zou, “Mixture-of-agents enhances large language model capabilities,” *arXiv preprint arXiv:2406.04692*, 2024.
- [25] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 8634–8652.
- [26] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe *et al.*, “Self-refine: Iterative refinement with self-feedback,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 46 534–46 594.
- [27] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, “Direct preference optimization: Your language model is secretly a reward model,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 53 728–53 741, 2023.
- [28] J. C.-Y. Chen, S. Saha, and M. Bansal, “Reconcile: Round-table conference improves reasoning via consensus among diverse llms,” *arXiv preprint arXiv:2309.13007*, 2023.
- [29] J. Piao, Y. Yan, J. Zhang, N. Li, J. Yan, X. Lan *et al.*, “Agentsocty: Large-scale simulation of llm-driven generative agents advances understanding of human behaviors and society,” *arXiv preprint arXiv:2502.08691*, 2025.
- [30] K. Zhang, R. Liu, X. Zhu, K. Tian, S. Zeng, G. Jia *et al.*, “Marti: A framework for multi-agent llm systems reinforced training and inference,” 2025.
- [31] M. Zhuge, W. Wang, L. Kirsch, F. Faccio, D. Khizbullin, and J. Schmidhuber, “GPTSwarm: Language agents as optimizable graphs,” in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024, pMLR.
- [32] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez *et al.*, “Routellm: Learning to route llms with preference data,” *arXiv preprint arXiv:2406.18665*, 2024.
- [33] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan *et al.*, “Dspy: Compiling declarative language model calls into self-improving pipelines,” *arXiv preprint arXiv:2310.03714*, 2023.
- [34] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge *et al.*, “Openhands: An open platform for ai software developers as generalist agents,” *arXiv preprint arXiv:2407.16741*, vol. 3, 2024.
- [35] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang *et al.*, “Metagpt: Meta programming for a multi-agent collaborative framework,” *arXiv preprint arXiv:2308.00352*, 2024.
- [36] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li *et al.*, “Chatdev: Communicative agents for software development,” *arXiv preprint arXiv:2307.07924*, 2024.
- [37] C. Gao, X. Lan, Z. Lu, J. Mao, J. Piao, H. Wang *et al.*, “S3: Social-network simulation system with large language model-empowered agents,” *arXiv preprint arXiv:2307.14984*, 2023.
- [38] W. Hua, L. Fan, L. Li, K. Mei, J. Ji, Y. Ge *et al.*, “War and peace (waragent): Large language model-based multi-agent simulation of world wars,” *arXiv preprint arXiv:2311.17227*, 2023.
- [39] T. Azfar, K. Huang, A. Tracy, S. Misiewicz, C. Liu, and R. Ke, “Traffic co-simulation framework empowered by infrastructure camera sensing and reinforcement learning,” *arXiv preprint arXiv:2412.03925*, 2024.
- [40] J. Li, Y. Lai, W. Li, J. Ren, M. Zhang, X. Kang *et al.*, “Agent hospital: A simulacrum of hospital with multi-agent cooperation for solving medical tasks,” *arXiv preprint arXiv:2405.02957*, 2024.
- [41] H. Li, Y. Shi, S. Lin, X. Gu, H. Lian, X. Wang *et al.*, “Swe-debate: Competitive multi-agent debate for software issue resolution,” *arXiv preprint arXiv:2507.23348*, 2025.
- [42] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar *et al.*, “Webarena: A realistic web environment for building autonomous agents,” *arXiv preprint arXiv:2307.13854*, 2023.
- [43] T. Xie, D. Zhang, J. Chen, X. Li, S. Zhao, R. Cao *et al.*, “Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 34 488–34 516, 2024.
- [44] C. Rawles, S. Clinckemahillie, Y. Chang, J. Waltz, G. Lau, M. Fair *et al.*, “Androidworld: A dynamic benchmarking environment for autonomous agents,” *arXiv preprint arXiv:2405.14573*, 2024.
- [45] J. S. Chan, N. Chowdhury, O. Jaffe, J. Aung, D. Sherburn, E. Mays *et al.*, “MLE-bench: Evaluating machine learning agents on machine learning engineering,” *arXiv preprint arXiv:2410.07095*, 2024.
- [46] J. P. Agapiou, A. S. Vezhnevets, E. A. Duñez-Guzmán, J. Matyas, Y. Mao, P. Suneag *et al.*, “Melting pot 2.0,” *arXiv preprint arXiv:2211.13746*, 2023.
- [47] K. Zhu, H. Du, Z. Hong, X. Yang, S. Guo, Z. Wang *et al.*, “Multiagent-bench: Evaluating the collaboration and competition of llm agents,” pp. 8580–8622, 2025.
- [48] Y. Cui and H. Du, “MAD-spear: A conformity-driven prompt injection attack on multi-agent debate systems,” *arXiv preprint arXiv:2507.13038*, 2025.
- [49] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 104 420–104 447, 2024.
- [50] Z. Zhang, S. Cui, Y. Lu, J. Zhou, J. Yang, H. Wang *et al.*, “Agent-safetybench: Evaluating the safety of llm agents,” 2025.
- [51] X. Li, S. Wang, S. Zeng, Y. Wu, and Y. Yang, “A survey on llm-based multi-agent systems: workflow, infrastructure, and challenges,” *Vicinagearth*, vol. 1, no. 1, p. 9, 2024.
- [52] B. Yan, X. Zhang, L. Zhang, L. Zhang, Z. Zhou, D. Miao *et al.*, “Beyond self-talk: A communication-centric survey of LLM-based multi-agent systems,” *arXiv preprint arXiv:2502.14321*, 2025.
- [53] C. Gao, X. Lan, N. Li, Y. Yuan, J. Ding, Z. Zhou *et al.*, “Large language models empowered agent-based modeling and simulation: A survey and perspectives,” *Humanities and Social Sciences Communications*, vol. 11, no. 1, pp. 1–24, 2024.
- [54] S. Han, K. Rao, A. Ettinger, L. Jiang, B. Y. Lin, N. Lambert *et al.*, “Wildguard: Open one-stop moderation tools for safety risks, jailbreaks, and refusals of llms,” *arXiv preprint arXiv:2406.18495*, 2024.
- [55] Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones *et al.*, “Constitutional ai: Harmlessness from ai feedback,” *arXiv preprint arXiv:2212.08073*, 2022.
- [56] D. Qiyan, X. Bai, K. Chen, Y. Wang, L. Nie, and M. Zhang, “Efficient safety alignment of large language models via preference re-ranking and representation-based reward modeling,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 31 156–31 171.
- [57] R. Jiang, K. Chen, X. Bai, Z. He, J. Li, M. Yang *et al.*, “A survey on human preference learning for aligning large language models,” *ACM Computing Surveys*, vol. 58, no. 6, pp. 1–39, 2025.
- [58] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin *et al.*, “Training language models to follow instructions with human feedback,” *Advances in neural information processing systems*, vol. 35, pp. 27 730–27 744, 2022.
- [59] Y.-M. Tseng, Y.-C. Huang, T.-Y. Hsiao, W.-L. Chen, C.-W. Huang, Y. Meng *et al.*, “Two tales of persona in LLMs: A survey of role-playing and personalization,” in *Findings Assoc. Comput. Linguistics: EMNLP 2024*, Miami, FL, USA, 2024, pp. 16 612–16 631.
- [60] Z. Zhang, R. A. Rossi, B. Kveton, Y. Shao, D. Yang, H. Zamani *et al.*, “Personalization of large language models: A survey,” *arXiv preprint arXiv:2411.00027*, 2024.
- [61] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang *et al.*, “Lora: Low-rank adaptation of large language models,” *ICLR*, vol. 1, no. 2, p. 3, 2022.
- [62] X. L. Li and P. Liang, “Prefix-tuning: Optimizing continuous prompts for generation,” *arXiv preprint arXiv:2101.00190*, 2021.
- [63] B. Lester, R. Al-Rfou, and N. Constant, “The power of scale for parameter-efficient prompt tuning,” in *Proc. Conf. Empir. Methods Nat. Lang. Process. (EMNLP)*, Punta Cana, Dominican Republic, 2021, pp. 3045–3059.
- [64] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection,” *arXiv preprint arXiv:2302.12173*, 2023.
- [65] Z. Zhang, X. Bo, C. Ma, R. Li, X. Chen, Q. Dai *et al.*, “A survey on the memory mechanism of large language model-based agents,” *ACM Transactions on Information Systems*, vol. 43, no. 6, pp. 1–47, 2025.
- [66] J. S. Park, J. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” in *Proceedings of the 36th annual acm symposium on user interface software and technology*, 2023, pp. 1–22.
- [67] Z. Pan, Q. Wu, H. Jiang, M. Xia, X. Luo, J. Zhang *et al.*, “Llmllingua-2: Data distillation for efficient and faithful task-agnostic prompt compression,” *arXiv preprint arXiv:2403.12968*, 2024.
- [68] H. Jiang, Q. Wu, X. Luo, D. Li, C.-Y. Lin, Y. Yang *et al.*, “Longllmlingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression,” *arXiv preprint arXiv:2310.06839*, 2023.

- [69] Q. Wang, Y. Fu, Y. Cao, S. Wang, Z. Tian, and L. Ding, "Recursively summarizing enables long-term dialogue memory in large language models," *Neurocomputing*, vol. 639, p. 130193, 2025.
- [70] W. Wang, L. Dong, H. Cheng, X. Liu, X. Yan, J. Gao *et al.*, "Augmenting language models with long-term memory," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, arXiv:2306.07174.
- [71] J. Suárez, P. Isola, K. W. Choe, D. Bloomin, H. X. Li, N. Pinnaraju *et al.*, "Neural mmo 2.0: a massively multi-task addition to massively multi-agent learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 50 094–50 104.
- [72] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai *et al.*, "Agentbench: Evaluating LLMs as agents," *arXiv preprint arXiv:2308.03688*, 2023.
- [73] K. Valmeekam, M. Marquez, S. Sreedharan, and S. Kambhampati, "On the planning abilities of large language models-a critical investigation," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 75 993–76 005.
- [74] A. Zhou, K. Yan, M. Shlapentokh-Rothman, H. Wang, and Y.-X. Wang, "Language agent tree search unifies reasoning acting and planning in language models," *arXiv preprint arXiv:2310.04406*, 2023.
- [75] X. Li, K. Chen, Y. Long, X. Bai, Y. Xu, and M. Zhang, "Generator-assistant stepwise rollback framework for large language model agent," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2025, pp. 17 683–17 700.
- [76] A. Mohammadshahi, A. R. Shaikh, and M. Yazdani, "Routoo: Learning to route to large language models effectively," *arXiv preprint arXiv:2401.13979*, 2024.
- [77] Z. Li, S. Xu, K. Mei, W. Hua, B. Rama, O. Raheja *et al.*, "Autoflow: Automated workflow generation for large language model agents," *arXiv preprint arXiv:2407.12821*, 2024.
- [78] S. Fan, X. Cong, Y. Fu, Z. Zhang, S. Zhang, Y. Liu *et al.*, "WorkflowLLM: Enhancing workflow orchestration capability of large language models," *arXiv preprint arXiv:2411.05451*, 2024.
- [79] Z. Guo, S. Cheng, H. Wang, S. Liang, Y. Qin, P. Li *et al.*, "Stable-toolbench: Towards stable large-scale benchmarking on tool learning of large language models," *arXiv preprint arXiv:2403.07714*, 2024.
- [80] S. G. Patil, H. Mao, F. Yan, C. C.-J. Ji, V. Suresh, I. Stoica *et al.*, "The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models," in *Forty-second International Conference on Machine Learning*, 2025.
- [81] S. Ye, H. Shi, D. Shih, H. Yun, T. Roosta, and T. Shu, "Realwebassist: A benchmark for long-horizon web assistance with real-world users," *arXiv preprint arXiv:2504.10445*, 2025.
- [82] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan *et al.*, "SWE-agent: Agent-computer interfaces enable automated software engineering," *Advances in Neural Information Processing Systems*, vol. 37, pp. 50 528–50 652, 2024.
- [83] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press *et al.*, "Swe-bench: Can language models resolve real-world github issues?" *arXiv preprint arXiv:2310.06770*, 2023.
- [84] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Chormanski *et al.*, "RT-2: Vision-language-action models transfer web knowledge to robotic control," in *Conference on Robot Learning*, 2024.
- [85] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David *et al.*, "Do as i can, not as i say: Grounding language in robotic affordances," *arXiv preprint arXiv:2204.01691*, 2022.
- [86] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair *et al.*, "Openvla: An open-source vision-language-action model," *arXiv preprint arXiv:2406.09246*, 2024.
- [87] W. Chen, Y. Su, J. Zuo, C. Yang, C. Yuan, C.-M. Chan *et al.*, "Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors in agents," *arXiv preprint arXiv:2308.10848*, 2023.
- [88] O. Topsakal and T. C. Akinci, "Creating large language model applications utilizing langchain: A primer on developing llm apps fast," in *International conference on applied engineering and natural sciences*, vol. 1, no. 1, 2023, pp. 1050–1056.
- [89] Z. Hou, J. Tang, and Y. Wang, "Halo: Hierarchical autonomous logic-oriented orchestration for multi-agent llm systems," *arXiv preprint arXiv:2505.13516*, 2025.
- [90] K. Mei, W. Xu, S. Lin, and Y. Zhang, "Omnirouter: Budget and performance controllable multi-llm routing," *arXiv preprint arXiv:2502.20576*, 2025.
- [91] B. Qiao, L. Li, X. Zhang, S. He, Y. Kang, C. Zhang *et al.*, "Taskweaver: A code-first agent framework," *arXiv preprint arXiv:2311.17541*, 2023.
- [92] X. Pan, D. Gao, Y. Xie, Y. Chen, Z. Wei, Y. Li *et al.*, "Very large-scale multi-agent simulation in agentscope," *arXiv preprint arXiv:2407.17789*, 2024.
- [93] W. Chen, Z. You, R. Li, Y. Guan, C. Qian, C. Zhao *et al.*, "Internet of agents: Weaving a web of heterogeneous agents for collaborative intelligence," *arXiv preprint arXiv:2407.07061*, 2024.
- [94] A. Shrimal, S. Kanagaraj, K. Biswas, S. Raghuraman, A. Nediyanath, Y. Zhang *et al.*, "MARCO: Multi-agent real-time chat orchestration," *arXiv preprint arXiv:2410.21784*, 2024.
- [95] J. Chen, S. Yuan, R. Ye, B. P. Majumder, and K. Richardson, "Put your money where your mouth is: Evaluating strategic planning and execution of LLM agents in an auction arena," *arXiv preprint arXiv:2310.05746*, 2024.
- [96] K. A. Dubey, Z. Feng, R. Kidambi, A. Mehta, and D. Wang, "Auctions with llm summaries," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM, 2024, pp. 713–722.
- [97] K. Agrawal, V. Teo, J. J. Vazquez, S. Kunnavakkam, V. Srikanth, and A. Liu, "Evaluating LLM agent collusion in double auctions," *arXiv preprint arXiv:2507.01413*, 2025.
- [98] B. Zhang, C. Li, G. Chen, and Z. Dong, "Large language model assisted optimal bidding of BESS in FCAS market: An AI-agent based approach," *arXiv preprint arXiv:2406.00974*, 2024.
- [99] R. M. Aratchige and W. M. K. S. Ilmini, "LLMs working in harmony: A survey on the technological aspects of building effective LLM-based multi-agent systems," *arXiv preprint arXiv:2504.01963*, 2025.
- [100] B. Pan, J. Lu, K. Wang, L. Zheng, Z. Wen, Y. Feng *et al.*, "Agentcoord: Visually exploring coordination strategy for llm-based multi-agent collaboration," *arXiv preprint arXiv:2404.11943*, 2024.
- [101] Y. Song, G. Wang, S. Li, and B. Y. Lin, "The good, the bad, and the greedy: Evaluation of LLMs should not ignore non-determinism," *arXiv preprint arXiv:2407.10457*, 2024.
- [102] S. Abdelnabi, A. Goma, S. Sivaprasad, L. Schönherr, and M. Fritz, "LLM-deliberation: Evaluating LLMs with interactive multi-agent negotiation games," *arXiv preprint arXiv:2309.17234*, 2023.
- [103] G. Zhang, Y. Yue, Z. Li, S. Yun, G. Wan, K. Wang *et al.*, "Cut the crap: An economical communication pipeline for LLM-based multi-agent systems," *arXiv preprint arXiv:2410.02506*, 2024.
- [104] Z. Yang, Z. Zhang, Z. Zheng, Y. Jiang, Z. Gan, Z. Wang *et al.*, "Oasis: Open agent social interaction simulations with one million agents," *arXiv preprint arXiv:2411.11581*, 2024.
- [105] M. Mastouri, E. Ksontini, and W. Kessentini, "Making REST APIs agent-ready: From OpenAPI to MCP servers for tool-augmented LLMs," *arXiv preprint arXiv:2507.16044*, 2025.
- [106] K.-T. Tran, D. Dao, M.-D. Nguyen, Q.-V. Pham, B. O'Sullivan, and H. D. Nguyen, "Multi-agent collaboration mechanisms: A survey of LLMs," *arXiv preprint arXiv:2501.06322*, 2025.
- [107] S. Luo, W. Shao, Y. Yao, J. Xu, M. Liu, Q. Li *et al.*, "Privacy in llm-based recommendation: Recent advances and future directions," *arXiv preprint arXiv:2406.01363*, 2024.
- [108] J. Richens and T. Everitt, "Robust agents learn causal world models," *arXiv preprint arXiv:2402.10877*, 2024.
- [109] B. Wang, W. He, S. Zeng, Z. Xiang, Y. Xing, J. Tang *et al.*, "Unveiling privacy risks in llm agent memory," *arXiv preprint arXiv:2502.13172*, 2025.
- [110] A. Seabra, C. Cavalcante, J. Nepomuceno, L. Lago, N. Ruberg, and S. Lifschitz, "Dynamic multi-agent orchestration and retrieval for multi-source question-answer systems using large language models," *arXiv preprint arXiv:2412.17964*, 2024.
- [111] S. Geng, M. Josifoski, M. Peyrard, and R. West, "Grammar-constrained decoding for structured nlp tasks without finetuning," *arXiv preprint arXiv:2305.13971*, 2023.
- [112] C. Ma, J. Zhang, Z. Zhu, C. Yang, Y. Yang, Y. Jin *et al.*, "Agentboard: An analytical evaluation board of multi-turn llm agents," *Advances in neural information processing systems*, vol. 37, pp. 74 325–74 362, 2024.
- [113] L. Song, J. Liu, J. Zhang, S. Zhang, A. Luo, S. Wang *et al.*, "Adaptive in-conversation team building for language model agents," *arXiv preprint arXiv:2405.19425*, 2024.
- [114] Z. Guan, X. Kong, F. Zhong, and Y. Wang, "Richelieu: Self-evolving llm-based agents for ai diplomacy," *Advances in Neural Information Processing Systems*, vol. 37, pp. 123 471–123 497, 2024.
- [115] T. Xia, Z. He, T. Ren, Y. Miao, Z. Zhang, Y. Yang *et al.*, "Measuring bargaining abilities of llms: A benchmark and a buyer-enhancement method," *arXiv preprint arXiv:2402.15813*, 2024.
- [116] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving factuality and reasoning in language models through multiagent debate," in *Forty-first International Conference on Machine Learning*, 2023.

- [117] Z. Liu, W. Yao, J. Zhang, L. Yang, Z. Liu, J. Tan *et al.*, “Agentlite: A lightweight library for building and advancing task-oriented llm agent system,” *arXiv preprint arXiv:2402.15538*, 2024.
- [118] K. Zhang, W. Yao, Z. Liu, Y. Feng, Z. Liu, R. Murthy *et al.*, “Diversity empowers intelligence: Integrating expertise of software engineering agents,” *arXiv preprint arXiv:2408.07060*, 2024.
- [119] Q. Wang, Z. Wang, Y. Su, H. Tong, and Y. Song, “Rethinking the bounds of llm reasoning: Are multi-agent discussions the key?” *arXiv preprint arXiv:2402.18272*, 2024.
- [120] Y. Song, F. Xu, S. Zhou, and G. Neubig, “Beyond browsing: Api-based web agents,” *arXiv preprint arXiv:2410.16464*, 2024.
- [121] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman *et al.*, “Eureka: Human-level reward design via coding large language models,” *arXiv preprint arXiv:2310.12931*, 2023.
- [122] R. G. Smith, “The contract net protocol: High-level communication and control in a distributed problem solver,” *IEEE Transactions on computers*, vol. 29, no. 12, pp. 1104–1113, 1980.
- [123] J. Wei, K. Shuster, A. Szlam, J. Weston, J. Urbanek, and M. Komeili, “Multi-party chat: Conversational agents in group settings with humans and models,” *arXiv preprint arXiv:2304.13835*, 2023.
- [124] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, “Autocoderover: Autonomous program improvement,” in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 1592–1604.
- [125] X. Wang, Y. Chen, L. Yuan, Y. Zhang, Y. Li, H. Peng *et al.*, “Executable code actions elicit better LLM agents,” in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, 2024, pp. 50 208–50 232.
- [126] Y. Meng, M. Xia, and D. Chen, “Simpo: Simple preference optimization with a reference-free reward,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 124 198–124 235, 2024.
- [127] Y. Shi, M. Wang, Y. Cao, H. Lai, J. Lan, X. Han *et al.*, “Aime: Towards fully-autonomous multi-agent framework,” *arXiv preprint arXiv:2507.11988*, 2025.
- [128] Z. Chen, Y. Deng, H. Yuan, K. Ji, and Q. Gu, “Self-play fine-tuning converts weak language models to strong language models,” *arXiv preprint arXiv:2401.01335*, 2024.
- [129] C. Sun, S. Huang, and D. Pompili, “LLM-based multi-agent reinforcement learning: Current and future directions,” *arXiv preprint arXiv:2405.11106*, 2024.
- [130] J. Liao, M. Wen, J. Wang, and W. Zhang, “MARFT: Multi-agent reinforcement fine-tuning,” *arXiv preprint arXiv:2504.16129*, 2025.
- [131] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou *et al.*, “Large language models as optimizers,” in *The Twelfth International Conference on Learning Representations*, 2023.
- [132] R. Pryzant, D. Iter, J. Li, Y. T. Lee, C. Zhu, and M. Zeng, “Automatic prompt optimization with “gradient descent” and beam search,” *arXiv preprint arXiv:2305.03495*, 2023.
- [133] L. Chen, M. Zaharia, and J. Zou, “Frugalgpt: How to use large language models while reducing cost and improving performance,” *arXiv preprint arXiv:2305.05176*, 2023.
- [134] G. Infantes, S. Roussel, P. Pereira, A. Jacquet, and E. Benazera, “Learning to solve job shop scheduling under uncertainty,” vol. 14797, pp. 329–345, 2024.
- [135] Y. Liu, S. Sun, G. Shen, X. V. Wang, M. Wiktorsson, and L. Wang, “An auction-based approach for multi-agent uniform parallel machine scheduling with dynamic jobs arrival,” *Engineering*, vol. 35, pp. 32–45, 2024.
- [136] Z. Zhao, D. Tang, C. Liu, L. Wang, Z. Zhang, H. Zhu *et al.*, “A large language model-based multi-agent manufacturing system for intelligent shopfloors,” *Advanced Engineering Informatics*, vol. 69, p. 103888, 2025.
- [137] Y. Xia, D. Dittler, N. Jazdi, H. Chen, and M. Weyrich, “Llm experiments with simulation: Large language model multi-agent system for simulation model parametrization in digital twins,” in *2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA)*, 2024, pp. 1–4.
- [138] L. Xu, S. Mak, M. Minaricova, and A. M. Brintrup, “On implementing autonomous supply chains: A multi-agent system approach,” *Computers in Industry*, vol. 161, p. 104120, 2024.
- [139] B. Kim, J. G. Kim, and S. Lee, “A multi-agent reinforcement learning model for inventory transshipments under supply chain disruption,” *IIE Transactions*, vol. 56, no. 7, pp. 715–728, 2024.
- [140] N. Kotecha and A. del Rio Chanona, “Leveraging graph neural networks and multi-agent reinforcement learning for inventory control in supply chains,” *Computers & Chemical Engineering*, p. 109111, 2025.
- [141] X. Yang, Z. Liu, W. Jiang, C. Zhang, L. Zhao, L. Song *et al.*, “A versatile multi-agent reinforcement learning benchmark for inventory management,” *arXiv preprint arXiv:2306.07542*, 2023.
- [142] B. Zhang, W. J. Tan, W. Cai, and A. N. Zhang, “Information sharing and confidentiality: a federated multi-agent reinforcement learning approach for supply chain coordination,” *International Journal of Production Research*, 2025.
- [143] Y. Yu, Z. Yao, H. Li, Z. Deng, Y. Cao, Z. Chen *et al.*, “Fincon: A synthesized llm multi-agent system with conceptual verbal reinforcement for enhanced financial decision making,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 137 010–137 045.
- [144] Y. Xiao, E. Sun, D. Luo, and W. Wang, “Tradingagents: Multi-agents LLM financial trading framework,” *arXiv preprint arXiv:2412.20138*, 2024.
- [145] Z. Wang, C. Ventre, and M. Polukarov, “ARL-based multi-action market making with hawkes processes and variable volatility,” in *Proceedings of the 5th ACM International Conference on AI in Finance*, 2024, pp. 437–444.
- [146] J. Chen, M. Zou, Z. Wang, Q. Wang, D. Sun, C. Zhang *et al.*, “Finhear: Human expertise and adaptive risk-aware temporal reasoning for financial decision-making,” *arXiv preprint arXiv:2506.09080*, 2025.
- [147] B. Li, W. Wu, Z. Tang, L. Shi, J. Yang, J. Li *et al.*, “Devbench: A comprehensive benchmark for software development,” *CoRR*, 2024.
- [148] T. Liu, C. Xu, and J. J. McAuley, “Repobench: Benchmarking repository-level code auto-completion systems,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024, arXiv:2306.03091.
- [149] P. Ma, R. Ding, S. Wang, S. Han, and D. Zhang, “Demonstration of insightpilot: An llm-empowered automated data exploration system,” *arXiv preprint arXiv:2304.00477*, 2023.
- [150] Z. Xu, S. Du, Y. Qi, C. Xu, C. Yuan, and J. Guo, “Chartbench: A benchmark for complex visual reasoning in charts,” *arXiv preprint arXiv:2312.15915*, 2023.
- [151] Z. Gu, X. Zhu, Y. Cai, H. Shen, X. Chen, Q. Wang *et al.*, “Agentgroupchat-v2: Divide-and-conquer is what llm-based multi-agent system need,” *arXiv preprint arXiv:2506.15451*, 2025.
- [152] N. Li, C. Gao, M. Li, Y. Li, and Q. Liao, “EconAgent: Large language model-empowered agents for simulating macroeconomic activities,” 2024.
- [153] S. Li, T. Azfar, and R. Ke, “Chatsumo: Large language model for automating traffic scenario generation in simulation of urban mobility,” *arXiv preprint arXiv:2409.09040*, 2024.
- [154] J. Light, M. Cai, S. Shen, and Z. Hu, “Avalonbench: Evaluating llms playing the game of avalon,” *arXiv preprint arXiv:2310.05036*, 2023.
- [155] J. Pasuksmit, W. Takerngsaksiri, P. Thongtanunam, C. Tantithamthorn, R. Zhang, S. Wang *et al.*, “Human-in-the-loop software development agents: Challenges and future directions,” in *2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*. IEEE, 2025, pp. 342–352.
- [156] V. Dibia, J. Chen, G. Bansal, S. Syed, A. Fournay, E. Zhu *et al.*, “Autogen studio: A no-code developer tool for building and debugging multi-agent systems,” *arXiv preprint arXiv:2408.15247*, 2024.
- [157] Z. Chen, E. Chason, N. Mladenovski, E. Wilson, K. Mullen, S. Martini *et al.*, “Sim911: Towards effective and equitable 9-1-1 dispatcher training with an LLM-enabled simulation,” in *Proc. 39th AAAI Conf. Artif. Intell. (AAAI)*, Philadelphia, PA, USA, 2025, AAAI Conf. on Artificial Intelligence, AI for Social Impact Track.
- [158] S. Schmidgall, R. Ziaei, C. Harris, E. Reis, J. Jopling, and M. Moor, “Agentclinic: A multi-agent framework for simulating outpatient clinic workflows,” *arXiv preprint arXiv:2405.07960*, 2024.
- [159] J. Wu, X. Liang, X. Bai, and Z. Chen, “Surgbox: Agent-driven operating room sandbox with surgery copilot,” in *Proceedings of the 2024 IEEE International Conference on Big Data (BigData)*. IEEE, 2024, pp. 2041–2048.
- [160] X. H. Lü, Z. Kasner, and S. Reddy, “Weblinx: Real-world website navigation with multi-turn dialogue,” *arXiv preprint arXiv:2402.05930*, 2024.
- [161] B. Yang, X. He, H. Gao, Y. Cao, X. Li, and D. Hsu, “Codeagents: A token-efficient framework for codified multi-agent reasoning in llms,” *arXiv preprint arXiv:2507.03254*, 2025.
- [162] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari *et al.*, “Why do multi-agent systems fail?” *arXiv preprint arXiv:2503.13657*, 2025.
- [163] Z. Liu, X. Bai, K. Chen, X. Chen, X. Li, Y. Xiang *et al.*, “A survey on the feedback mechanism of llm-based ai agents,” in *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence, IJCAI-25*, 2025, pp. 10 582–10 592.